ASSIGNMENT – 5

CSA1006

Software Engineering

Name: P.Poojitha

Register No:192472349

1. Explain Technical Debt. What are the major causes of technical debt, and how can organizations minimize it during software evolution?

Ans:

Introduction

Technical debt refers to the future cost or additional effort that arises when developers choose a quick or easy software solution instead of a better long-term solution.

Similar to financial debt, technical debt may help a project move faster initially, but it creates additional work later.

Example

Suppose developers need to add a feature quickly. Instead of designing a proper database structure, they use a temporary solution:

Quick Solution → Faster Delivery → More Maintenance → Technical Debt

Later, changing or extending the system becomes difficult because other parts of the software depend on the temporary solution.

Major Causes of Technical Debt

1. Tight Project Deadlines

When there is pressure to release software quickly, developers may skip proper design, testing, or documentation.

2. Poor Software Design

Incorrect architecture, tightly coupled modules, and duplicated code make future modifications difficult.

3. Lack of Testing

Insufficient unit, integration, and regression testing allows defects to remain in the system.

4. Outdated Technologies

Old programming languages, libraries, frameworks, or dependencies can become difficult to maintain and integrate.

5. Poor Documentation

When the system is not properly documented, future developers spend more time understanding the code.

6. Code Duplication

Repeated code increases maintenance effort because the same logic must be modified in multiple places.

7. Frequent Requirement Changes

Continuous changes in business requirements can result in patches and temporary solutions.

8. Lack of Code Reviews

Without peer review, poor coding practices and design problems can remain unnoticed.

How Organizations Can Minimize Technical Debt

1. Use Good Software Architecture

Organizations should design modular, scalable, and maintainable systems from the beginning.

2. Perform Regular Code Reviews

Developers should review each other's code to identify design problems, duplication, and bad practices.

3. Automate Testing

Automated unit, integration, and regression tests help detect defects early.

4. Refactor Regularly

Refactoring means improving the internal structure of code without changing its external behavior.

5. Keep Dependencies Updated

Libraries and frameworks should be updated regularly to improve security, compatibility, and maintainability.

6. Maintain Proper Documentation

Architecture, APIs, important decisions, and system behavior should be documented.

7. Track Technical Debt

Technical debt items should be recorded in the project backlog and prioritized along with normal development tasks.

8. Follow Coding Standards

Common naming conventions, design principles, and coding standards improve consistency.

Technical Debt Management During Software Evolution

Software continuously changes through:

New Requirements → Modifications → New Features → Maintenance → Evolution

During each stage, organizations should allocate time for testing, refactoring, documentation, and architectural improvements.

Technical debt cannot always be completely avoided because some shortcuts may be necessary for business reasons. However, organizations can control and reduce technical debt through good architecture, code reviews, automated testing, regular refactoring, documentation, and continuous maintenance. This helps software remain reliable, secure, and maintainable throughout its evolution.

2. Explain how AI/ML and Edge Computing are transforming modern software engineering. Provide one practical example for each.

Ans:

Modern software engineering is changing rapidly because of Artificial Intelligence/Machine Learning (AI/ML) and Edge Computing. They help developers build software that is more intelligent, automated, responsive, and scalable.

1. AI/ML in Modern Software Engineering

Artificial Intelligence (AI) enables software to perform tasks that normally require human intelligence, while Machine Learning (ML) enables systems to learn patterns from data and make predictions.

How AI/ML Transform Software Engineering

1. Automated Code Generation

AI tools can assist developers in generating code from natural-language descriptions.

2. Bug Detection

ML models can identify patterns associated with software defects and help developers find potential bugs.

3. Intelligent Testing

AI can generate test cases and identify areas of the application that require additional testing.

4. Predictive Maintenance

ML can analyze system logs and historical data to predict possible failures before they occur.

5. Improved Security

AI can detect unusual behavior and identify potential security threats.

6. Personalized Applications

ML allows applications to provide recommendations based on user behavior.

Practical Example – E-Commerce Recommendation System

An online shopping application can use ML to analyze:

- Previous purchases
- Product searches
- User preferences
- Browsing history

The ML model predicts products that the customer may be interested in.

User Activity

↓

ML Model

↓

Pattern Analysis

↓

Product Recommendation

↓

Personalized Shopping

Benefit: The system provides personalized recommendations and can improve customer experience and sales.

2. Edge Computing in Modern Software Engineering

Edge Computing processes data close to the location where it is generated instead of sending all data to a centralized cloud server.

Traditional approach:

IoT Device → Internet → Cloud → Response

Edge computing:

IoT Device → Edge Device → Immediate Response

How Edge Computing Transforms Software Engineering

1. Low Latency

Data can be processed locally, reducing communication delay.

2. Faster Decision Making

Applications can make decisions without waiting for a cloud response.

3. Reduced Bandwidth Usage

Only important or processed information needs to be sent to the cloud.

4. Improved Reliability

Some operations can continue even when internet connectivity is poor.

5. Better Privacy

Sensitive data can sometimes be processed locally rather than being sent to a remote server.

Practical Example – Smart Traffic Management

Cameras installed at a traffic intersection capture vehicle

movement. Instead of sending every video frame to the cloud:

Traffic Camera

↓

Edge Device

↓

Vehicle Detection

↓

Traffic Density Analysis

↓

Traffic Signal Adjustment

The edge device can immediately detect traffic congestion and adjust traffic signals.

Benefit: Faster response, lower network usage, and improved traffic management.

AI/ML vs Edge Computing

AI/ML	Edge Computing
Makes software intelligent	Processes data near the source
Learns patterns from data	Reduces communication delay
Supports prediction and automation	Supports real-time processing
Used for recommendations, detection, prediction	Used in IoT, smart cities, autonomous systems

AI/ML makes software more intelligent and predictive, while Edge Computing makes software faster and more responsive. Together, these technologies support the development of modern applications such as smart healthcare, autonomous vehicles, smart cities, and intelligent IoT systems.

3. Case Study – A company deploys dozens of microservices in the cloud and notices that resource usage and energy consumption are increasing. Explain three sustainable DevOps practices that can reduce energy consumption and cloud cost.

Ans:

1. Auto-Scaling and Right-Sizing

Cloud resources should be adjusted according to actual workload.

For example:

Low Traffic → Fewer Instances

High Traffic → More Instances

Unused virtual machines and containers should be stopped or removed.

Benefits:

- Reduces unnecessary CPU and memory usage
- Reduces cloud costs
- Reduces energy consumption

2. Container Optimization

Docker containers can be optimized by:

- Using lightweight base images
- Removing unnecessary dependencies
- Limiting CPU and memory usage
- Running only required services

This improves resource utilization and reduces the infrastructure required to run microservices.

3. Continuous Monitoring and Energy-Aware Deployment

DevOps teams can continuously monitor:

- CPU utilization
- Memory usage
- Number of running containers
- Cloud resource consumption
- Application performance

Tools such as monitoring dashboards can help identify underutilized resources.

Applications can also be deployed in energy-efficient cloud regions or during periods of lower energy demand, when appropriate.

Benefits:

- Prevents resource wastage
- Reduces cloud expenditure
- Improves infrastructure efficiency
- Supports environmentally sustainable software development

Sustainable DevOps combines software delivery with efficient resource management. Auto-scaling and right-sizing, container optimization, and continuous monitoring can reduce unnecessary resource usage, cloud costs, and energy consumption while maintaining application performance.